PG220 : Manipulation 2

Hicham LAKHLEF

10 septembre 2026

1 Introduction

Pour vous faciliter les manipulations de cette séance, sachez que vous pouvez placer plusieurs classes dans le même fichier Java. Par contre, pour pouvoir lancer le programme facilement, il faut que le nom du fichier soit celui de la classe qui contient la méthode point d'entrée.

2 Héritage

Dans cette manipulation, nous allons tester le fonctionnement de la méthode `toString()` qui est définie dans la classe `Object` (et donc disponible sur n'importe quel objet). Tout d'abord, consultez la javadoc de cette classe. Créez ensuite un programme de test qui crée un objet vide et affiche le résultat de la méthode `toString()`. Ensuite, créez un string et affichez le résultat de la méthode `toString()`. On vérifie de cette manière que la méthode a été redéfinie, le comportement n'est plus le même. Créez aussi une classe `Point` (voir manipulation 1) et affichez le résultat de la méthode `toString()`. Est-ce que le code est le même que sur celle d'`Object` à votre avis ?

3 Redéfinition de méthode

Maintenant, nous allons réaliser un premier exemple de redéfinition de méthode. Pour cela, vous allez définir les deux classes `ElementRepere` et `Point`. Bien entendu, `Point` hérite de `ElementRepere`. Vous allez maintenant redéfinir la célèbre méthode `public String toString()` dans ces deux classes. Il faut que :

- `toString()` de `ElementRepere` affiche : *Je suis un ElementRepere.*
- `toString()` de `Point` affiche : *Je suis un ElementRepere. Je suis aussi un Point.*

L'objectif est bien entendu d'avoir le moins de duplication de code possible.

4 Héritage et constructeurs

Nous allons maintenant manipuler l'héritage et son effet sur les constructeurs. Pour cela, vous allez définir les deux classes `ElementRepere` et `Point`. Définissez en outre un attribut `String titre` dans `ElementRepere`, et les attributs correspondants aux coordonnées dans `Point`. Vous allez maintenant définir les constructeurs de ces deux classes qui construisent des objets ayant une valeur pour chaque attribut. L'objectif est bien entendu d'avoir le moins de duplication de code possible.

5 Héritage, code statique et ordre d'exécution

Nous allons maintenant analyser les effets des attributs statiques et de l'héritage sur le comportement d'un programme. Attendez-vous à des surprises. Tout d'abord, il faut récupérer le code suivant. Lisez maintenant le code avec attention et essayez de trouver quelle est la séquence de chaînes qui va s'afficher sur la console après exécution.

```
class JavaMadness {
    public static void main(String[] args) {
        System.out.println("Debut du point d'entree.");
        System.out.println("Declaration d'un objet de type Point.");
        Point p;
        System.out.println("Access a une propriete statique");
        VerboseString s = Point.sX;
        System.out.println("Creation d'un objet de type Point.");
        p = new Point(); // seule création d'objet du point d'entree.
    }
}

class VerboseString {
    public VerboseString(String s) {
        System.out.println(s);
    }
}

class ElementAvecTitre {
    public static VerboseString sTitle = new VerboseString("sTitle cree statiquement.");

    public VerboseString title = new VerboseString("title cree dynamiquement.");

    public ElementAvecTitre() {
        System.out.println("Appel du constructeur de ElementAvecTitre.");
    }
}
```

```
class Point extends ElementAvecTitre {
    public static VerboseString sX = new VerboseString("sX cree statiquement.");

    public VerboseString x = new VerboseString("x cree dynamiquement.");

    public Point() {
        System.out.println("Appel du constructeur de Point.");
    }
}
```

Compilez le programme (`javac JavaMadness.java` suivi de `java JavaMadness`). Exécutez-le. Essayez de comprendre pourquoi la séquence de chaînes s'affiche dans l'ordre que vous avez obtenu. N'hésitez pas à poser des questions à l'intervenant si vous ne comprenez pas.

6 Modification de l'ordre d'exécution

6.1 Étape 1

Supprimez l'instruction `VerboseString s = Point.sX;` dans la méthode `main` et recompilez. Pourquoi la séquence de chaînes affichée sur la console a changé ?

6.2 Étape 2

Supprimez aussi l'appel à `p = new Point();` dans la méthode `main` et recompilez. Pourquoi la séquence de chaînes affichée sur la console a changé ?